

단어의 의미를 분포에서 학습하기

공기행렬 → PMI/SVD → Word2Vec → GloVe → fastText

분포 가설

비슷한 문맥에 등장하는 단어는 비슷한 의미·기능을 갖는 경향이 있다.

- **관측** — “학생이 __을 신청했다”의 빈칸에 장학금·수강·기숙사가 반복된다.
- **학습** — 주변 단어를 예측하거나 공기 통계를 압축해 가까운 벡터를 만든다.

주의

분포는 의미 전체가 아니다. 사실성, 논리, 인과, 가치 판단을 자동으로 보장하지 않는다.

학습목표

1. window, co-occurrence, PMI/PPMI, SVD를 설명한다.
2. CBOW와 Skip-gram의 입력·목표를 비교한다.
3. negative sampling이 full softmax 비용을 줄이는 원리를 설명한다.
4. Word2Vec, GloVe, fastText의 학습 신호와 오류를 비교한다.
5. Hands-On LLM의 Gensim/추천 예제를 평가 가능한 실험으로 확장한다.

공기행렬(co-occurrence matrix)

중심 단어 w 주변 c 칸 안에 문맥 단어 u 가 몇 번 등장했는지 센다.

$$C_{w,u} = \#\{u \text{ appears within window of } w\}$$

중심\문맥	학생	신청	장학금	교수
수강	12	18	2	4
장학금	10	16	25	1
연구	5	3	1	19

- 작은 window: 문법·기능 유사성
- 큰 window: 주제·관련성
- 좌우 문맥을 구분할 수도 있다.

PMI와 PPMI

두 사건이 독립일 때보다 얼마나 자주 함께 나타나는가?

$$\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$$

$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0)$$

- 빈도만 쓰면 “하다”, “있다” 같은 고빈도 단어가 지배한다.
- PMI는 기대 빈도 대비 과대표현을 강조한다.
- 매우 희귀한 쌍의 PMI가 과도하게 커질 수 있어 smoothing/min-count가 필요하다.

SVD/LSA: 큰 희소행렬을 낮은 차원으로

$$X \approx U_k \Sigma_k V_k^\top$$

- X : 단어-문맥 또는 문서-단어 행렬
- k : 남길 잠재 차원
- $U_k \Sigma_k$: 단어/문서의 밀집 표현
- **직관** — 함께 변하는 많은 단어 축을 소수의 잠재 방향으로 압축한다. 잡음 완화와 동의어 연결이 가능하지만 선형 구조라는 한계가 있다.

논문 읽기

신경망 임베딩 이전에도 “분포 통계 + 저차원 압축”은 강력한 표현학습이었다.

Word2Vec의 두 과제

- **CBOW** — 주변 단어들 → 중심 단어 예측 학생이 [MASK] 신청했다 → 장학금을
- **Skip-gram** — 중심 단어 → 주변 단어 예측 장학금 → 학생, 신청, 지급

정의

정적 단어 임베딩: 어휘 항목마다 하나의 벡터를 저장한다. “배를 먹다”와 “배를 타다”의 배가 같은 벡터라는 구조적 한계가 있다.

Skip-gram의 목표

중심 단어 w_t 로 주변 단어를 예측:

$$\max_{\theta} \sum_t \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} \mid w_t)$$

full softmax:

$$P(o \mid i) = \frac{\exp(\mathbf{v}'_o{}^\top \mathbf{v}_i)}{\sum_{w \in V} \exp(\mathbf{v}'_w{}^\top \mathbf{v}_i)}$$

어휘 $|V|$ 가 크면 분모 계산이 비싸다. 그래서 hierarchical softmax 또는 negative sampling을 사용한다.

Negative sampling

관측된 양성 쌍 (w, c) 는 가깝게, 잡음 분포에서 뽑은 k 개 음성 n_i 는 멀게:

$$\log \sigma(\mathbf{v}'_c{}^\top \mathbf{v}_w) + \sum_{i=1}^k \log \sigma(-\mathbf{v}'_{n_i}{}^\top \mathbf{v}_w)$$

- 전체 어휘 분모 대신 선택된 몇 개 쌍만 갱신한다.
- 음성 분포와 k 가 학습 결과에 영향을 준다.
- 자주 등장하는 단어를 subsampling해 계산량과 편향을 줄인다.

출처: Mikolov et al. (2013), “Distributed Representations of Words and Phrases...” · [arXiv:1310.4546](https://arxiv.org/abs/1310.4546)

논문 핵심 그림을 다시 읽는 법

그림: CBOW는 주변 문맥으로 중심 단어를, Skip-gram은 중심 단어로 주변 단어를 예측한다.

교재 연결

Chapter 2의 **Word Embeddings Beyond LLMs** 예제는 이 예측 과제로 학습된 벡터를 내려받아 유사도와 추천에 사용한다. 화살표 방향은 곧 학습 데이터의 입력-정답 정의다.

출처: Mikolov et al. (2013), "Efficient Estimation of Word Representations in Vector Space," Figure 1 · [원 논문](#)

GloVe: 전역 공기 통계를 회귀한다

$$J = \sum_{i,j} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

- X_{ij} : 전체 말뭉치에서 단어 i, j 의 공기 횟수
- f : 지나치게 희귀/빈번한 쌍의 영향 제어
- Word2Vec의 국소 예측과 달리 **전역 행렬 통계**를 명시적으로 사용

정의

둘은 구현은 다르지만 “분포에서 단어 관계를 저차원 기하로 압축한다”는 목표를 공유한다.

출처: Pennington et al. (2014), “GloVe.” · [ACL Anthology](#)

fastText: 단어를 문자 n-gram의 합으로

$$\mathbf{v}_{word} = \sum_{g \in \mathcal{G}(word)} \mathbf{z}_g$$

예: 장학금 → 경계 기호를 포함한 여러 문자 n-gram

- 미등록 단어도 n-gram 조합으로 근사 가능
- 활용·철자 변이가 많은 언어와 희귀어에 유리할 수 있음
- 비슷한 철자지만 다른 의미인 단어가 과도하게 가까워질 수 있음

논문 읽기

한국어에서 형태·문자 정보의 이점은 corpus와 tokenizer에 따라 다르므로 intrinsic + downstream 평가가 필요하다.

벡터 연산과 analogy

고전 예시의 형태:

$$\mathbf{v}(king) - \mathbf{v}(man) + \mathbf{v}(woman) \approx \mathbf{v}(queen)$$

설명 가능한 부분:

- 어떤 관계가 반복되는 방향으로 정렬될 수 있다.
- 최근접 이웃/analogy가 구조를 탐색하는 도구가 된다.

설명하면 안 되는 부분:

- 모든 개념 관계가 선형이라는 결론
- 한 예시 성공을 일반 지능으로 확대
- 편향된 관계를 자연·보편적 사실로 해석

편향도 벡터에 압축된다

분포 학습은 말뭉치의 사회적 연관성을 반영한다.

- 직업-성별, 지역-계층 등 해로운 연관이 이웃과 방향에 나타날 수 있다.
- bias “제거”는 과업 성능과 다른 집단의 관계를 바꿀 수 있다.
- 정적 벡터의 유사도만으로 사람·집단을 분류하는 것은 위험하다.

주의

평가 원칙: 전체 평균뿐 아니라 집단별 false positive/negative, 사용 맥락, 피해의 비대칭을 기록한다.

임베딩 기반 데이터 과학 · 03주차

Hands-On LLM 연결

Gensim 결과를 추천 시스템의 증거로 바꾸기

국립목포대학교 컴퓨터학부 · Hands-On LLM 연계

원본 코드의 핵심 파라미터

```
from gensim.models import Word2Vec

model = Word2Vec(
    sentences=tokenized_corpus,
    vector_size=100,
    window=5,
    min_count=3,
    sg=1,           # 1: skip-gram, 0: CBOW
    negative=10,
    epochs=20,
    seed=42,
)
```

파라미터	바꾸면 무엇이 달라지는가
window	구문적/주제적 문맥 범위
min_count	희귀어 보존 vs 잡음/메모리
vector_size	용량·비용·과적합 가능성

평균 단어 벡터로 문서 만들기

```
def mean_embed(tokens, kv):  
    kept = [kv[t] for t in tokens if t in kv]  
    return np.mean(kept, axis=0) if kept else np.zeros(kv.vector_size)
```

이 간단한 기준선이 잃는 것:

- 어순: “학생이 교수를 평가” vs “교수가 학생을 평가”
- 부정: “신청 가능” vs “신청 불가”
- 단어 중요도: 모든 토큰 동일 가중치
- 다의성: 문맥과 무관한 하나의 단어 벡터

실습

개선: TF-IDF 가중 평균, SIF, 문장 Transformer와 비교하고 동일 test query로 평가한다.

추천 데모를 실험으로 바꾸기

단계	데모	평가 가능한 실험
데이터	노래 몇 곡	train/test와 메타데이터 명시
표현	평균 벡터	TF-IDF, Word2Vec, SBERT 비교
결과	이웃 5개 관찰	정답/선호 라벨로 Recall@k
해석	“비슷하다”	장르·아티스트 편향, 다양성 분석

교재 연결

Hands-On LLM의 가치: 한 줄의 `most_similar` 가 끝이 아니라, 표현 선택이 추천 결과에 어떻게 전파되는지 관찰하는 출발점이다.

정적 임베딩은 끝났는가?

아니다. 다음 조건에서 여전히 유용할 수 있다.

- CPU/edge 환경, 매우 낮은 지연시간, 작은 메모리
- 토큰 수준 특성을 해석해야 하는 고전 ML 파이프라인
- 작은 도메인 corpus에 빠르게 적응하는 기준선
- 형태 정보가 중요한 희귀어 처리

최신 동향

현재 위치: 범용 semantic search의 중심은 문맥적 문장 임베딩으로 이동했지만, 정적 벡터는 비용-해석성 기준선이자 구성요소다.

수업 활동: window가 의미를 바꾼다

1. 같은 corpus로 **window=2** 와 **window=10** 모델을 학습한다.
2. 구문적 단어 5개, 주제적 단어 5개의 이웃을 저장한다.
3. 두 모델의 이웃 overlap@10을 계산한다.
4. downstream 문서 분류/검색 중 하나로 성능을 비교한다.

실습

결론 형식: “큰 window는 __ 관계를 강화했고, 이 변화가 __ 과업에서 지표를 __만큼 변화시켰다.”

형성평가

1. 분포 가설을 자신의 예로 설명하라.
2. PMI가 단순 빈도보다 공통어 영향을 줄이는 원리는?
3. CBOW와 Skip-gram의 입력/정답은?
4. negative sampling에서 false negative가 생기면 어떤 방향으로 학습되는가?
5. 평균 단어 벡터 문서 표현의 구조적 한계 두 가지는?

교재 연결

Notebook: [../notebooks/week03.ipynb](#) · seed, corpus, 파라미터, OOV 비율을 함께 기록한다.

핵심 정리

- 단어 임베딩은 주변 분포를 저차원 기하에 압축한다.
- 공기행렬/PPMI/SVD와 Word2Vec/GloVe는 서로 다른 계산으로 같은 큰 질문을 푼다.
- fastText는 subword 합으로 희귀어와 형태 변이를 다룬다.
- 정적 벡터는 빠르고 유용하지만 다의성·어순·부정·편향에 구조적 한계가 있다.
- 하이퍼파라미터가 “의미”의 종류를 바꾸므로 downstream 평가가 필수다.

핵심

다음 주: 같은 단어가 **문맥마다 다른 벡터**가 된다.

참고문헌과 원본 연결

- Mikolov et al. (2013), “Efficient Estimation of Word Representations in Vector Space.” [arXiv](#)
- Mikolov et al. (2013), “Distributed Representations of Words and Phrases...” [arXiv](#)
- Levy & Goldberg (2014), “Neural Word Embedding as Implicit Matrix Factorization.” [NeurIPS](#)
- Pennington et al. (2014), “GloVe.” [ACL](#)
- Bojanowski et al. (2017), “Enriching Word Vectors with Subword Information.” [TACL](#)
- Hands-On Large Language Models, Chapter 2. [upstream](#)